Module 05 — Collaborative Filtering Improvements

src/collaborative_filtering.py • 282 lines

In one sentence

Fixes the specific ways textbook collaborative filtering fails on real data.

1. What this module does

Basic collaborative filtering works, but it fails in predictable ways once the data looks like a real platform. This module addresses each failure directly.

The biggest problem is **popularity bias**. Because best-sellers appear in so many customers' histories, they get recommended to everyone, which makes them even more popular. Left alone, the system collapses into an expensive way of showing a top-sellers list, and 80% of the catalogue never gets seen.

The fix is a re-ranking layer that gently penalises over-exposed products and boosts under-exposed ones. Getting that balance right turned out to be much harder than it sounds - see the warnings below.

2. Concepts covered

Every idea this module uses, in plain terms.

Concept	In simple terms	Where it is used here
Popularity bias	Popular things get recommended, becoming more popular	The main problem this module fixes
Inverse propensity weighting	Down-weight things you see too often	The popularity penalty
Re-ranking	Reorder a list after scoring it	rerank_with_popularity_balance()
Two-stage retrieval	Shortlist first, then reorder	Retrieve top 200, then diversify
Min-max normalisation	Rescale scores to 0-1 before combining	Applied before any multiplication
Long-tail boost	Give rarely-seen products a small lift	1.15x multiplier
Implicit CF	Similarity from actions, not ratings	create_item_similarity_from_implicit()
Cold-start fallback	What to do with no history	score_items_for_new_user()
Accuracy vs diversity trade-off	Relevant results vs varied results	The alpha parameter

3. How it works, step by step

1. **Build item similarity from cart and purchase events** rather than ratings, so it works for every interaction.
2. **Precompute a penalty for every product**: 1 / (interactions + 1) raised to a small power. Heavily-exposed products get a smaller multiplier.
3. **Precompute a boost** of 1.15x for products in the long tail.

4. **When ranking: take the top 200 by relevance first.** Only then apply the penalty and boost inside that shortlist.
5. **Rescale scores to 0-1 before multiplying**, because the raw scores can be negative.
6. **For customers with no history**, fall back to their registration profile.

4. Key functions

Function	What it does
create_item_similarity_from_implicit()	Item similarity from cart/purchase
build_popularity_lookup()	Precomputes penalty and boost per product
min_max_normalize()	Rescales any score vector to 0-1
rerank_with_popularity_balance()	The two-stage re-ranker
get_cold_start_items()	Products with under 3 interactions
get_cold_start_users()	Customers with under 3 interactions
score_items_for_new_user()	Registration-based fallback

5. Inputs, outputs and how to run

	Detail
Inputs	implicit matrix, popularity features, predicted ratings
Outputs	implicit_item_similarity.pkl, popularity_lookup.pkl
Key setting	POPULARITY_PENALTY_ALPHA = 0.20
Runtime	About 40 seconds
Run with	python src/collaborative_filtering.py

Choosing the penalty strength (alpha) by measurement

alpha	Overlap with the uncorrected list	Verdict
0.00	100%	no correction at all
0.10	97%	barely noticeable
0.20	**92%**	**visible variety, relevance kept - chosen**
0.35	86%	starts costing real relevance

6. Key code

src/collaborative_filtering.py — build_popularity_lookup()

```
56 | def build_popularity_lookup(item_popularity_features: pd.DataFrame,
57 |                             penalty_alpha: float = POPULARITY_PENALTY_ALPHA) -> pd.DataFrame:
58 |     """Precompute the per-item re-ranking multipliers."""
59 |     lookup = item_popularity_features.copy()
60 |
61 |     # Power form, not 1/(1+log1p(count)): that spans a ~5x range, wider than
62 |     # the scores it multiplies, and collapses the ranking onto obscure items.
63 |     lookup["popularity_penalty"] = 1.0 / np.power(
64 |         lookup["interaction_count"] + 1.0, penalty_alpha
65 |     )
66 |     lookup["long_tail_boost"] = np.where(lookup["is_long_tail"] == 1, LONG_TAIL_BOOST, 1.00)
67 |
68 |     return lookup.set_index("item_id")
```

src/collaborative_filtering.py — rerank_with_popularity_balance()

```
106 | def rerank_with_popularity_balance(score_series: pd.Series, popularity_lookup: pd.DataFrame,
107 |                                    candidate_pool: int = CANDIDATE_POOL_SIZE) -> pd.DataFrame:
108 |     """Retrieve by relevance, then diversify within the retrieved pool."""
109 |     scores = score_series.astype(float)
110 |
111 |     # Pool first: applied catalogue-wide, the penalty promotes irrelevant
112 |     # obscure items above genuinely relevant ones.
113 |     if candidate_pool is not None and len(scores) > candidate_pool:
114 |         scores = scores.nlargest(candidate_pool)
115 |
116 |     # Normalise before multiplying: SVD scores are signed, and a negative
117 |     # score times a penalty below 1 moves UP, inverting the correction.
118 |     recommendation_df = pd.DataFrame({
119 |         "raw_score": min_max_normalize(scores),
120 |         "original_score": scores,
121 |     })
122 |
123 |     available = [c for c in RERANK_COLUMNS if c in popularity_lookup.columns]
124 |     recommendation_df = recommendation_df.join(popularity_lookup[available], how="left")
125 |
126 |     for column, default in RERANK_DEFAULTS.items():
127 |         if column in recommendation_df.columns:
128 |             recommendation_df[column] = recommendation_df[column].fillna(default)
129 |         else:
130 |             recommendation_df[column] = default
131 |
132 |     recommendation_df["adjusted_score"] = (
133 |         recommendation_df["raw_score"]
134 |         * recommendation_df["popularity_penalty"]
135 |         * recommendation_df["long_tail_boost"]
136 |     )
137 |
138 |     return recommendation_df.sort_values("adjusted_score", ascending=False)
```

Two hard-won design decisions are encoded in this one function.

7. Things to remember

Mistake 1: the obvious penalty formula destroyed the results

The natural choice is 1 / (1 + log(count)). It was tried first and it is a trap: across this catalogue it varies by about 5x, which is *wider* than the spread of the relevance scores it multiplies. So the penalty completely took over the ranking. Measured, it pushed the average exposure of the top 10 results from 518 interactions down to **1**, and 100% of results were long-tail. The system had stopped recommending relevant things and started recommending obscure things for the sake of it.

Mistake 2: a sign bug that reversed the correction

SVD produces **negative** scores for many products, because it runs on the mean-centred matrix. Multiplying a negative number by a penalty below 1 makes it *bigger* (closer to zero). So every product that should have been pushed down was being pushed up. Rescaling to 0-1 first fixes it.

The structural fix: retrieve, then re-rank

Applying the diversity correction across the whole catalogue lets an obscure product the customer has no interest in beat a genuinely relevant one, purely for being obscure. Taking the top 200 by relevance *first* means everything in the final list is relevant first and diverse second. This is the standard retrieve-then-rank split used in real systems.

8. Likely questions

Q. What is popularity bias and why does it matter commercially?

A. Popular products appear in more histories, so they get recommended more, so they become more popular. The catalogue collapses onto a shrinking head. Commercially that means most of your inventory becomes invisible and never sells. The popularity baseline in this project has just 1.3% catalogue coverage.

Q. How did you choose alpha = 0.20?

A. By measuring, not guessing. I compared the top 10 results before and after correction at several alpha values. At 0.20 the lists still overlap 92%, so relevance is intact, but there is visible promotion of under-exposed products. At 0.35 the overlap drops to 86% and it starts costing real accuracy.

Q. Why apply the correction to only 200 candidates?

A. Because relevance has to come first. Across the full catalogue of 2,264 products, the penalty will happily promote something the customer has no affinity for at all. Restricting to a relevance-ranked shortlist means the diversity adjustment only reorders things that were already worth showing.